

# **Memory Management**

# Introduction

- Linux takes advantage of Intel's segmentation and paging circuits to translate logical addresses into physical ones.
- Some portion of RAM is permanently assigned to the kernel and used to store both the kernel code and the static kernel data structures.
- The remaining part of the RAM is called dynamic memory. It is a valuable resource, needed not only by the processes but also by the kernel itself.
- In fact, the performance of the entire system depends on how efficiently dynamic memory is managed.
- Therefore, all current multitasking operating systems try to optimize the use of dynamic memory, assigning it only when it is needed and freeing it as soon as possible.

# Page Frame Management

- The Intel Pentium processor can use two different page frame sizes: 4 KB and 4 MB.
- Linux adopts the smaller 4 KB page frame size as the standard memory allocation unit.
- This makes things simpler for two reasons:
  - The paging circuitry automatically checks whether the page being addressed is contained in some page frame; furthermore, each page frame is hardware-protected through the flags included in the Page Table entry that points to it. By choosing a 4 KB allocation unit, the kernel can directly determine the memory allocation unit associated with the page where a page fault exception occurs.
  - The 4 KB size is a multiple of most disk block sizes, so transfers of data between main memory and disks are more efficient. Yet this smaller size is much more manageable than the 4 MB size.
- The kernel must keep track of the current status of each page frame.
- For instance, it must be able to distinguish the page frames used to contain pages belonging to processes from those that contain kernel code or kernel data structures.
- Similarly, it must be able to determine whether a page frame in dynamic memory is free or not. This sort of state information is kept in an array of descriptors, one for each page frame.

- The descriptors of type struct page have the following format:

```
typedef struct page {  
    struct page *next;  
    struct page *prev;  
    struct inode *inode;  
    unsigned long offset;  
    struct page *next_hash;  
    atomic_t count;  
    unsigned long flags;  
    struct wait_queue *wait;  
    struct page **pprev_hash;  
    struct buffer_head * buffers;  
} mem_map_t;
```

- Some of the fields are -
  - count - Set 0 to if the corresponding page frame is free; set to a value greater than if the page frame has been assigned to one or more processes or if it is used for some kernel data structures.
  - prev , next - Used to insert the descriptor in a doubly linked circular list. The meaning of these fields depends on the current use of the page frame.
  - flags - An array of up to 32 flags describing the status of the page frame. For each PG\_xyz flag, a corresponding PageXYZ macro has been defined to read or set its value.

| Table 6-1. Flags Describing the Status of a Page Frame |                                                                                                                                          |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Flag Name                                              | Meaning                                                                                                                                  |
| PG_decr_after                                          | See <a href="#">Section 16.4.3</a> in <a href="#">Chapter 16</a> .                                                                       |
| PG_dirty                                               | Not used.                                                                                                                                |
| PG_error                                               | An I/O error occurred while transferring the page.                                                                                       |
| PG_free_after                                          | See <a href="#">Section 15.1.1</a> in <a href="#">Chapter 15</a> .                                                                       |
| PG_DMA                                                 | Usable by ISA DMA (see text).                                                                                                            |
| PG_locked                                              | Page cannot be swapped out.                                                                                                              |
| PG_referenced                                          | Page frame has been accessed through the hash table of the page cache (see <a href="#">Section 14.2</a> in <a href="#">Chapter 14</a> ). |
| PG_reserved                                            | Page frame reserved to kernel code or unusable.                                                                                          |
| PG_skip                                                | Used on SPARC/SPARC64 architectures to "skip" some parts of the address space.                                                           |
| PG_Slab                                                | Included in a slab: see <a href="#">Section 6.2</a> later in this chapter.                                                               |
| PG_swap_cache                                          | Included in the swap cache; see <a href="#">Section 16.3</a> in <a href="#">Chapter 16</a>                                               |
| PG_swap_unlock_after                                   | See <a href="#">Section 16.4.3</a> in <a href="#">Chapter 16</a> .                                                                       |
| PG_uptodate                                            | Set after completing a read operation, unless a disk I/O error happened.                                                                 |



- Page frame descriptors are initialized by the **free\_area\_init( )** function, which acts on two parameters: **start\_mem** denotes the first linear address of the dynamic memory immediately after the kernel memory, while **end\_mem** denotes the last linear address of the dynamic memory plus 1.
- The **free\_area\_init( )** function also considers the `i386_endbase` variable, which stores the initial address of the reserved page frames.
- The function allocates a suitably sized memory area to `mem_map`.
- The function then initializes the area by setting all fields to 0, except for the flags fields, in which it sets the `PG_DMA` and `PG_reserved` flags.
- Subsequently, the `mem_init( )` function clears both the `PG_reserved` flag of the page frames, so they can be used as dynamic memory and the `PG_DMA` flags of all page frames having physical addresses greater than or equal to `0x1000000`.

- First, the **mem\_init( )** function determines the value of `num_physpages`, the total number of page frames present in the system. It then counts the number of page frames of type `PG_reserved`.
- Several symbols produced while compiling the kernel enable the function to count the number of page frames reserved for the hardware, kernel code, and kernel data and the number of page frames used during kernel initialization that can be successively released.
- Finally, **mem\_init( )** sets the count field of each page frame descriptor associated with the dynamic memory to 1 and calls the **free\_page( )** function.
- Since this function increments the value of the variable `nr_free_pages`, that variable will contain the total number of page frames in the dynamic memory at the end of the loop.



```
    mem_map = (mem_map_t *) start_mem;
    p = mem_map + MAP_NR(end_mem);
start_mem = ((unsigned long) p + sizeof(long) - 1) & ~(sizeof(long)-1);
memset(mem_map, 0, start_mem - (unsigned long) mem_map);
do {
    --p;
    p->count = 0;
    p->flags = (1 << PG_DMA) | (1 << PG_reserved);
} while (p > mem_map);
start_low_mem = PAGE_SIZE + PAGE_OFFSET;
num_physpages = MAP_NR(end_mem);
while (start_low_mem < i386_endbase) {
    clear_bit(PG_reserved,
&mem_map[MAP_NR(start_low_mem)].flags);
    start_low_mem += PAGE_SIZE;
}
```

```
while (start_mem < end_mem) {
    clear_bit(PG_reserved,
&mem_map[MAP_NR(start_mem)].flags);
    start_mem += PAGE_SIZE;
}
for (tmp = PAGE_OFFSET ; tmp < end_mem ; tmp += PAGE_SIZE) {
    if (tmp >= PAGE_OFFSET+0x1000000)
        clear_bit(PG_DMA, &mem_map[MAP_NR(tmp)].flags);
    if (PageReserved(mem_map+MAP_NR(tmp))) {
if (tmp >= (unsigned long) &_amp;text && tmp < (unsigned long) &_amp;edata)
        if (tmp < (unsigned long) &_amp;etext)
            codepages++;
else
            datapages++;
    }
```

```
        else if (tmp >= (unsigned long) &__init_begin && tmp < (unsigned long)
&__init_end)
            initpages++;
        else if (tmp >= (unsigned long) &__bss_start && tmp < (unsigned long)
start_mem)
            datapages++;
        else
            reservedpages++;
        continue;
    }

    mem_map[MAP_NR(tmp)].count = 1;
    free_page(tmp);
}
```

# Requesting and Releasing Page Frames

- The kernel allocates and initializes the data structures for page frame handling.
- Page frames can be requested by making use of four slightly differing functions and macros:
- `__get_free_pages(gfp_mask, order)` - Function used to request 2<sup>order</sup> contiguous page frames.
- `__get_dma_pages(gfp_mask, order)` - Macro used to get page frames suitable for DMA; it expands to:

`__get_free_pages(gfp_mask | GFP_DMA, order)`

- `__get_free_page(gfp_mask)` - Macro used to get a single page frame; it expands to:

`__get_free_pages(gfp_mask, 0)`

- `get_free_page(gfp_mask)` - Function that invokes:

`__get_free_page(gfp_mask)`

and then fills the page frame obtained with zeros.

- The parameter `gfp_mask` specifies how to look for free page frames. It consists of the following flags:
- `__GFP_WAIT` - Set if the kernel is allowed to discard the contents of page frames in order to free memory before satisfying the request.

- `__GFP_IO` - Set if the kernel is allowed to write pages to disk in order to free the corresponding page frames. (Since swapping can block the process in Kernel Mode, this flag must be cleared when handling interrupts or modifying critical kernel data structures.)
  - `__GFP_DMA` - Set if the requested page frames must be suitable for DMA.
  - `__GFP_HIGH` , `__GFP_MED` , `__GFP_LOW` - Specify the request priority.
- `__GFP_LOW` is usually associated with dynamic memory requests issued by User Mode processes, while the other priorities are associated with kernel requests.

**Table 6-2. Groups of Flag Values Used to Request Page Frames**

| Group Name              | <code>__GFP_WAIT</code> | <code>__GFP_IO</code> | Priority                |
|-------------------------|-------------------------|-----------------------|-------------------------|
| <code>GFP_ATOMIC</code> | 0                       | 0                     | <code>__GFP_HIGH</code> |
| <code>GFP_BUFFER</code> | 1                       | 0                     | <code>__GFP_LOW</code>  |
| <code>GFP_KERNEL</code> | 1                       | 1                     | <code>__GFP_MED</code>  |
| <code>GFP_NFS</code>    | 1                       | 1                     | <code>__GFP_HIGH</code> |
| <code>GFP_USER</code>   | 1                       | 1                     | <code>__GFP_LOW</code>  |

- Page frames can be released through any of the following three functions and macros:
- `free_pages(addr, order)` - This function checks the page descriptor of the page frame having physical address `addr`; if the page frame is not reserved (i.e., if the `PG_reserved` flag is equal to 0), it decrements the count field of the descriptor. If count becomes 0, it assumes that `2order` contiguous page frames starting from `addr` are no longer used. In that case, the function invokes `free_pages_ok( )` to insert the page frame descriptor of the first free page in the proper list of free page frames.
- `__free_page(p)` - Similar to the previous function, except that it releases the page frame whose descriptor is pointed to by parameter `p`.
- `free_page(addr)` - Macro used to release the page frame having physical address `addr`; it expands to `free_pages(addr,0)`.

# The Buddy System Algorithm

- The kernel must establish a robust and efficient strategy for allocating groups of contiguous page frames.
- In doing so, it must deal with a well-known memory management problem called external fragmentation : frequent requests and releases of groups of contiguous page frames of different sizes may lead to a situation in which several small blocks of free page frames are "scattered" inside blocks of allocated page frames. As a result, it may become impossible to allocate a large block of contiguous page frames, even if there are enough free pages to satisfy the request.
- There are essentially two ways to avoid external fragmentation:
  - Make use of the paging circuitry to map groups of noncontiguous free page frames into intervals of contiguous linear addresses.
  - Develop a suitable technique to keep track of the existing blocks of free contiguous page frames, avoiding as much as possible the need to split up a large free block in order to satisfy a request for a smaller one.

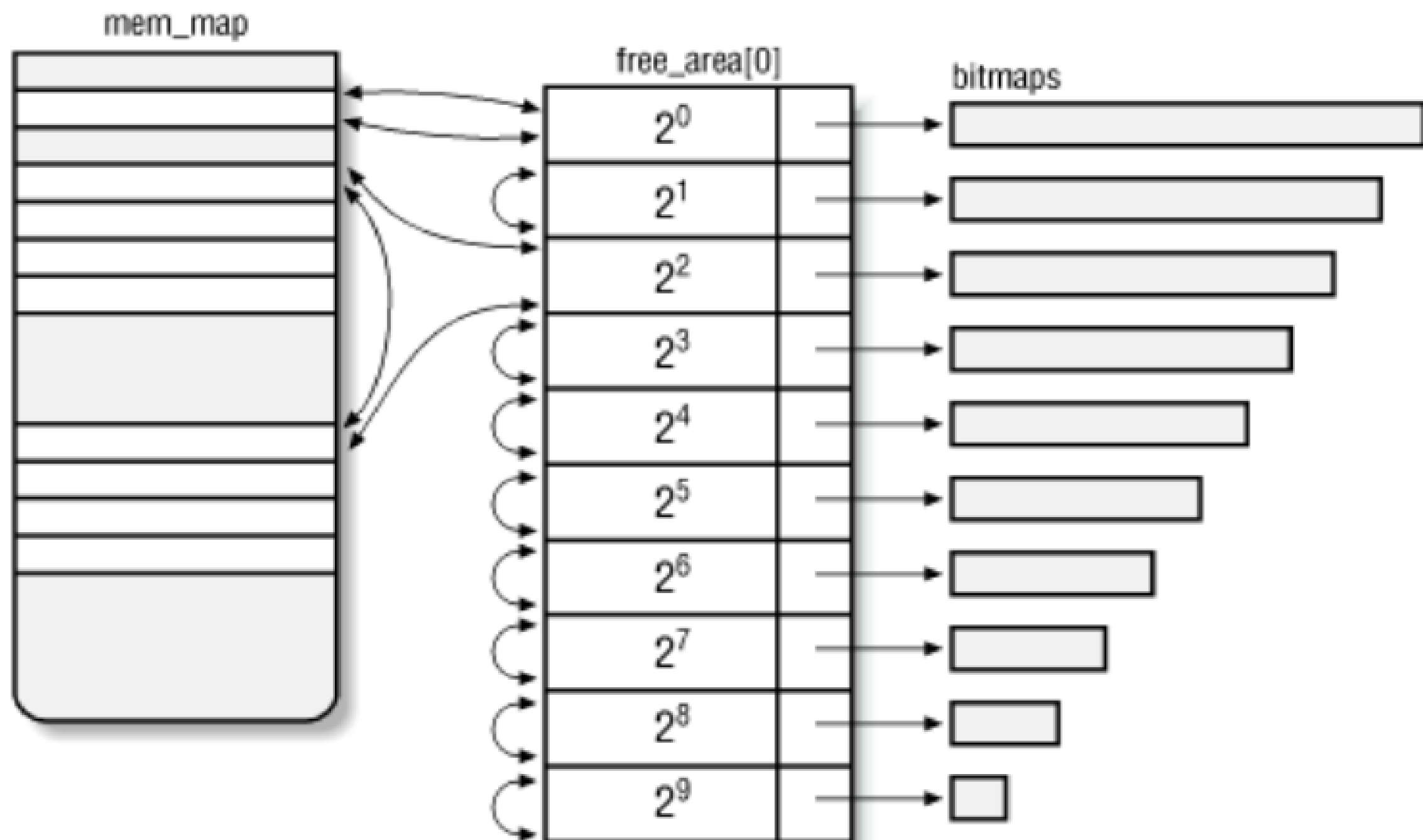
- The second approach is the one preferred by the kernel for two good reasons:
  - In some cases, contiguous page frames are really necessary, since contiguous linear addresses are not sufficient to satisfy the request. A typical example is a memory request for buffers to be assigned to a DMA processor.
- Since the DMA ignores the paging circuitry and accesses the address bus directly while transferring several disk sectors in a single I/O operation, the buffers requested must be located in contiguous page frames.
  - Even if contiguous page frame allocation is not strictly necessary, it offers the big advantage of leaving the kernel paging tables unchanged. What's wrong with modifying the page tables? As we know from Chapter 2, frequent page table modifications lead to higher average memory access times, since they make the CPU flush the contents of the translation lookaside buffers.
- The technique adopted by Linux to solve the external fragmentation problem is based on the well-known buddy system algorithm. All free page frames are grouped into 10 lists of blocks that contain groups of 1, 2, 4, 8, 16, 32, 64, 128, 256, and 512 contiguous page frames, respectively. The physical address of the first page frame of a block is a multiple of the group size: for example, the initial address of a 16-page-frame block is a multiple of  $16 \times 2^{12}$ .



- The reverse operation, releasing blocks of page frames, gives rise to the name of this algorithm. The kernel attempts to merge together pairs of free buddy blocks of size  $b$  into a single block of size  $2b$ . Two blocks are considered buddy if:
  - Both blocks have the same size, say  $b$ .
  - They are located in contiguous physical addresses.
  - The physical address of the first page frame of the first block is a multiple of  $2 \times b \times 2^{12}$ .
- The algorithm is iterative; if it succeeds in merging released blocks, it doubles  $b$  and tries again so as to create even bigger blocks.

# Data Structures

- Linux makes use of two different buddy systems: one handles the page frames suitable for ISA DMA, while the other one handles the remaining page frames. Each buddy system relies on the following main data structures:
- The `mem_map` array introduced previously.
- An array having 10 elements of type **`free_area_struct`**, one element for each group size. The variable **`free_area[0]`** points to the array used by the buddy system for the page frames that are not suitable for ISA DMA, while **`free_area[1]`** points to the array used by the buddy system for page frames suitable for ISA DMA.
- Ten binary arrays named `bitmaps`, one for each group size. Each buddy system has its own set of `bitmaps`, which it uses to keep track of the blocks it allocates.



# Allocating a block

- The `__get_free_pages()` function implements the buddy system strategy for allocating page frames.
- This function checks first whether there are enough free pages, that is, if `nr_free_pages` is greater than `freepages.min`.
- If not, it may decide to reclaim page frames. Otherwise, it goes on with the allocation by executing the code included in the `RMQUEUE_TYPE` macro:

```
if (!(gfp_mask & __GFP_DMA))  
    RMQUEUE_TYPE(order, 0);  
    RMQUEUE_TYPE(order, 1);
```

- The order parameter denotes the logarithm of the size of the requested block of free pages.
- The second parameter is the index into `free_area`, which is for non-DMA blocks and 1 for DMA blocks. So the code checks `gfp_mask` to see whether non-DMA blocks are allowed and, if so, tries to get blocks from that list (index 0), because it would be better to save DMA blocks for requests that really need them.
- If the page frames are successfully allocated, the code in the `RMQUEUE_TYPE` macro executes a return statement, thus terminating the `__get_free_pages()` function.
- Otherwise, the code in the `RMQUEUE_TYPE` macro is executed again with the second parameter equal to 1, that is, the memory allocation request is satisfied using page frames suitable for DMA.

- RMQUEUE\_TYPE updates the count field of the page descriptor associated with the selected block and executes a return instruction:

ret->count = 1;

return PAGE\_OFFSET + (map\_nr << PAGE\_SHIFT);

- As a result, the **\_\_get\_free\_pages( )** function returns the address of the block found.

# Freeing a Block

- The `free_pages_ok( )` function implements the buddy system strategy for freeing page frames. It makes use of three input parameters:
- `map_nr` - The page number of one of the page frames included in the block to be released.
- `order` - The logarithmic size of the block
- `type` - Equal to 1 if the page frames are suitable for DMA and 0 if they are not
- The function starts by declaring and initializing a few local variables:

# Memory Area Management

- This section deals with memory areas, that is, with sequences of memory cells having contiguous physical addresses and an arbitrary length.
- The buddy system algorithm adopts the page frame as the basic memory area. This is fine for dealing with relatively large memory requests, but how are we going to deal with requests for small memory areas, say a few tens or hundred of bytes?
- It would be wasteful to allocate a full page frame to store a few bytes. The correct approach is to introduce new data structures that describe how small memory areas are allocated within the same page frame. In doing so, we introduce a new problem called internal fragmentation. It is caused by a mismatch between the size of the memory request and the size of the memory area allocated to satisfy the request.
- A classical solution adopted by Linux 2.0 consists of providing memory areas whose sizes are geometrically distributed, i.e., the size depends on a power of 2 rather than on the size of the data to be stored.
- In this way, no matter what the memory request size is, we can ensure that the internal fragmentation is always smaller than 50%.
- Following this approach, Linux 2.0 creates 13 geometrically distributed lists of free memory areas whose sizes range from 32 to 131056 bytes.
- The buddy system is invoked both to obtain additional page frames needed to store new memory areas and conversely to release page frames that no longer contain memory areas. A dynamic list is used to keep track of the free memory areas contained in each page frame.

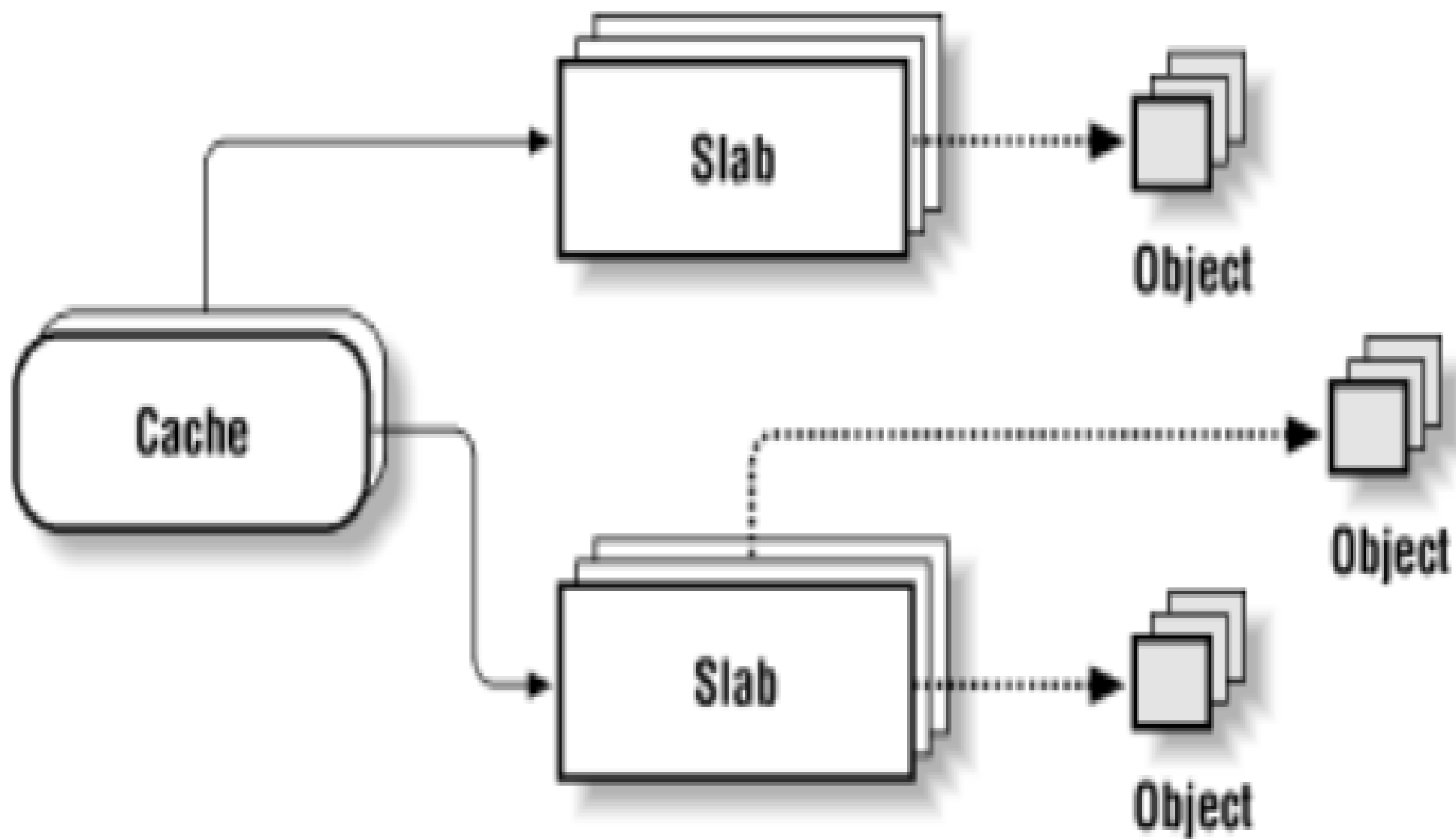
# The Slab Allocator

- Running a memory area allocation algorithm on top of the buddy algorithm is not efficient.
- Linux 2.2 reexamines the memory area allocation from scratch and comes out with some very clever improvements.
- The new algorithm is derived from the slab allocator schema developed in 1994 for the Sun Microsystem Solaris 2.4 operating system. It is based on the following premises:
- The type of data to be stored may affect how memory areas are allocated; for instance, when allocating a page frame to a User Mode process, the kernel invokes the **get\_free\_page( )** function, which fills the page with zeros. The concept of a slab allocator expands upon this idea and views the memory areas as objects consisting of both a set of data structures and a couple of functions or methods called the constructor and destructor : the former initializes the memory area while the latter deinitializes it. In order to avoid initializing objects repeatedly, the slab allocator does not discard the objects that have been allocated and then released but saves them in memory. When a new object is then requested, it can be taken from memory without having to be reinitialized.



- The kernel functions tend to request memory areas of the same type repeatedly. For instance, whenever the kernel creates a new process, it allocates memory areas for some fixed size tables such as the process descriptor, the open file object, and so on. When a process terminates, the memory areas used to contain these tables can be reused. Since processes are created and destroyed frequently, previous versions of the Linux kernel wasted time allocating and deallocating the page frames containing the same memory areas repeatedly. In Linux 2.2 they are saved in a cache and reused instead.
- Requests for memory areas can be classified according to their frequency. Requests of a particular size that are expected to occur frequently can be handled most efficiently by creating a set of special purpose objects having the right size, thus avoiding internal fragmentation. Therefore, sizes that are rarely encountered can be handled through an allocation scheme based on objects in a series of geometrically distributed sizes (such as the power-of-2 sizes used in Linux 2.0), even if this approach leads to internal fragmentation.
- There is another way in introducing objects whose sizes are not geometrically distributed: the initial addresses of the data structures are less prone to be concentrated on physical addresses whose values are a power of 2. This, in turn, leads to better performance by the processor hardware cache.
- Hardware cache performance creates an additional reason for limiting calls to the buddy system allocator as much as possible: every call to a buddy system function "dirty" the hardware cache, thus increasing the average memory access time

- The slab allocator groups objects into caches.
- Each cache is a "store" of objects of the same type.
- For instance, when a file is opened, the memory area needed to store the corresponding "open file" object is taken from a slab allocator cache named filp (for "file pointer").
- The slab allocator caches used by Linux may be viewed at runtime by reading the /proc/slabinfo file.
- The area of main memory that contains a cache is divided into slabs; each slab consists of one or more contiguous page frames that contain both allocated and free objects.
- The slab allocator never releases the page frames of an empty slab on its own.
- It would not know when free memory is needed, and there is no benefit to releasing objects when there is still plenty of free memory for new objects.
- Therefore, releases occur only when the kernel is looking for additional free page frames.



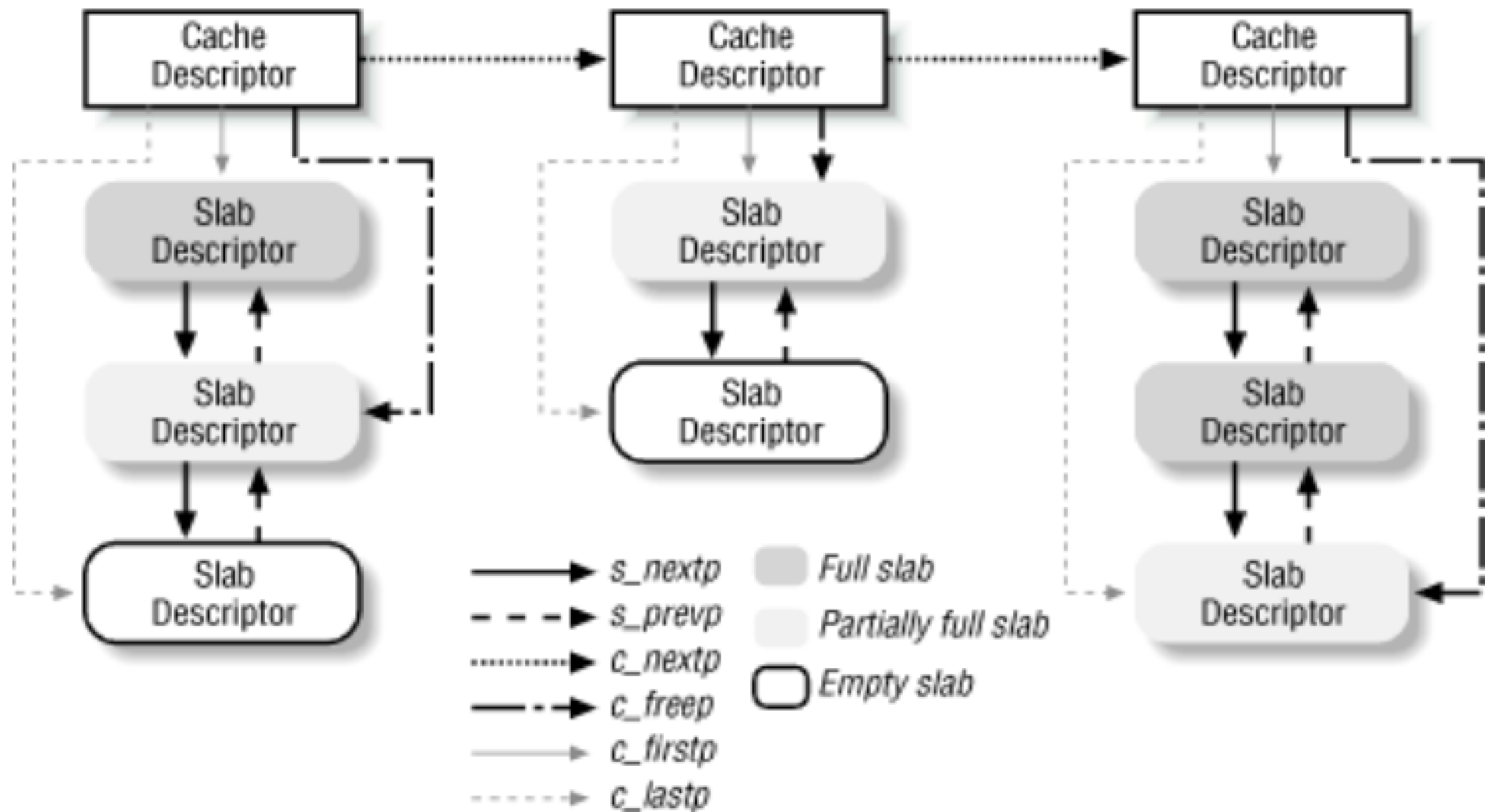
# Cache Descriptor

- Each cache is described by a table of type **struct kmem\_cache\_s**. The most significant fields of this table are:
- **c\_name** - Points to the name of the cache.
- **c\_firstp** , **c\_lastp** - Point, respectively, to the first and last slab descriptor of the cache. The slab descriptors of a cache are linked together through a doubly linked, circular, partially ordered list: the first elements of the list include slabs with no free objects, then come the slabs that include used objects along with at least one free object, and finally the slabs that include only free objects.
- **c\_freep** - Points to the **s\_nextp** field of the first slab descriptor that includes at least one free object.
- **c\_num** - Number of objects packed into a single slab. (All slabs of the cache have the same size.)
- **c\_offset** - Size of the objects included in the cache. (This size may be rounded up if the initial addresses of the objects must be memory aligned.)
- **c\_gfporder** - Logarithm of the number of contiguous page frames included in a single slab.
- **c\_ctor** , **c\_dtor** - Point, respectively, to the constructor and destructor methods associated with the cache objects. They are currently set to NULL, as stated earlier.

- `c_nextp` - Points to the next cache descriptor. All cache descriptors are linked together in a simple list by means of this field.
- `c_flags` - An array of flags that describes some permanent properties of the cache. There is, for instance, a flag that specifies which of two possible alternatives has been chosen to store the object descriptors in memory.
- `c_magic` - A magic number selected from a predefined set of values. Used to check both the current state of the cache and its consistency.

# Slab Descriptor

- Each slab of a cache has its own descriptor of type struct **kmem\_slab\_s**.
- Slab descriptors can be stored in two possible places, the choice depending normally on the size of the objects in the slab. If the object size is smaller than 512 bytes, the slab descriptor is stored at the end of the slab; otherwise, it is stored outside of the slab.
- The most significant fields of a slab descriptor are:
  - **s\_inuse** - Number of objects in the slab that are currently allocated.
  - **s\_mem** - Points to the first object (either allocated or free) inside the slab.
  - **s\_freep** - Points to the first free object (if any) in the slab.
  - **s\_nextp** , **s\_prevp** - Point, to the next and previous slab descriptor. The **s\_nextp** field of the last slab descriptor in the list points to the **c\_offset** field of the corresponding cache descriptor.
  - **s\_dma** - Flag set if the objects included in the slab can be used by the DMA processor.
  - **s\_magic** - Similar to the **c\_magic** field of the cache descriptor. It contains a magic number selected from a predefined set of values and is used to check both the current state of the slab and its consistency. The values of this field are different from those of the corresponding **c\_magic** field of the cache descriptor. The offset of **s\_magic** within the slab descriptor is equal to the offset of **c\_magic** with respect to **c\_offset** inside the cache descriptor; the checking routine relies on their being the same.



# Interfacing Slab Allocator with Buddy system

- When the slab allocator creates new slabs, it relies on the buddy system algorithm to obtain a group of free contiguous page frames.
- It invokes the **kmem\_getpages( )** function.
- The parameters used in **kmem\_getpages( )** function have the following meaning:
- **cachep** - Points to the cache descriptor of the cache that needs additional page frames (the number of required page frames is in the `cachep->c_gfporder` field)
- **flags** - Specifies how the page frame is requested (see Section 6.1.1 earlier in this chapter)
- **dma** - Points to a variable that is set to 1 by **kmem\_getpages( )** if the allocated page frames are suitable for ISA DMA.
- In the reverse operation, page frames assigned to a slab allocator can be released by invoking the **kmem\_freepages( )** function.
- The function releases the page frames, starting from the one having physical address `addr`, that had been allocated to the slab of the cache identified by `cachep`.



# Allocating a Slab to a Cache

- A newly created cache does not contain any slab and therefore no free objects. New slabs are assigned to a cache only when both of the following are true:
- A request has been issued to allocate a new object.
- The cache does not include any free object.
- When this occurs, the slab allocator assigns a new slab to the cache by invoking **kmem\_cache\_grow( )**. This function calls **kmem\_getpages( )** to obtain a group of page frames from the buddy system.
- It then calls **kmem\_cache\_slabmgmt( )** to get a new slab descriptor.
- Next, it calls **kmem\_cache\_init\_objs( )**, which applies the constructor method to all the objects contained in the new slab. It then calls **kmem\_slab\_link\_end( )**, which inserts the slab descriptor at the end of the cache slab list.
- The **kmem\_slab\_end** macro yields the address of the `c_offset` field of the corresponding cache descriptor.
- After inserting the new slab descriptor into the list, **kmem\_cache\_grow( )** loads the `next` and `prev` fields, respectively, of the descriptors of all page frames included in the new slab with the address of the cache descriptor and the address of the slab descriptor.
- This works correctly because the `next` and `prev` fields are used by functions of the buddy system only when the page frame is free, while page frames handled by the slab allocator functions are not free as far as the buddy system is concerned.

# Releasing a Slab from Cache

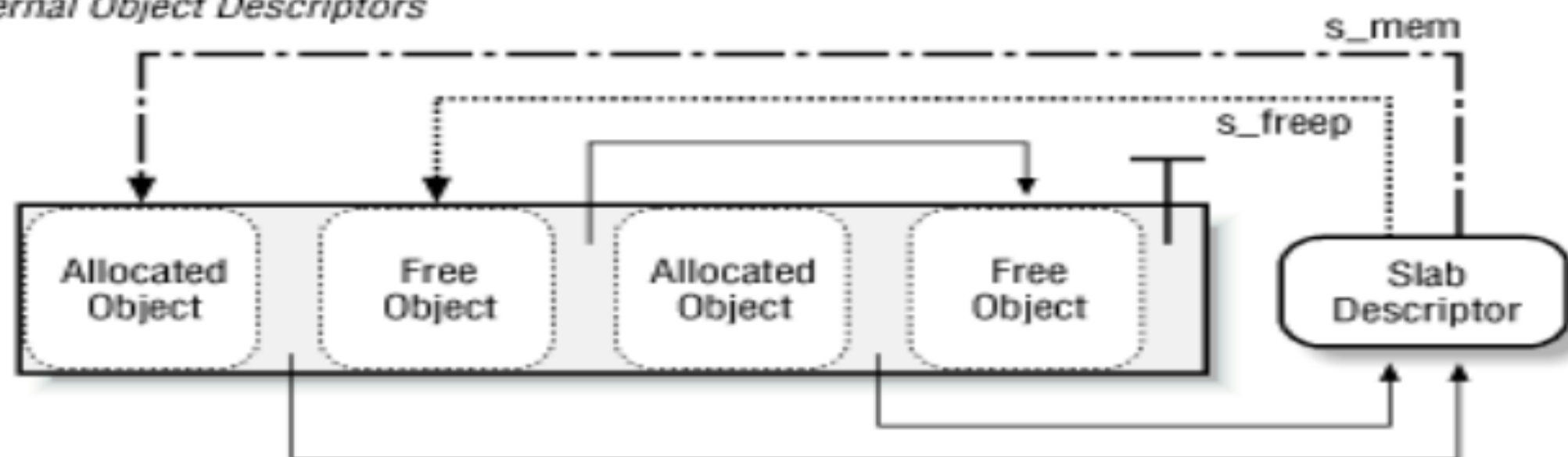
- The slab allocator never releases the page frames of an empty slab on its own. In fact, a slab is released only if both the following conditions hold:
- The buddy system is unable to satisfy a new request for a group of page frames.
- The slab is empty, that is, all the objects included in it are free.
- When the kernel looks for additional free page frames, it calls **try\_to\_free\_pages( )**; this function, in turn, may invoke **kmem\_cache\_reap( )**, which selects a cache that contains at least one empty slab. The **kmem\_slab\_unlink( )** function then removes the slab from the cache list of slabs.
- The slab—together with the objects in it—is destroyed by invoking **kmem\_slab\_destroy( )**.

- The function checks whether the cache has a destructor method for its objects in which case it applies the destructor to all the objects in the slab; the **objp** local variable keeps track of the currently examined object.
- Next, it calls **kmem\_freepages( )**, which returns all the contiguous page frames used by the slab to the buddy system.
- Finally, if the slab descriptor is stored outside of the slab , the function releases it from the cache of the slab descriptors.
- In order to avoid wasting memory space, the kernel must destroy all slabs in all caches created by a module before removing it.
- The **kmem\_cache\_shrink( )** function destroys all the slabs in a cache by invoking **kmem\_slab\_destroy( )** iteratively.
- The **c\_growing** field of the cache descriptor is used to prevent **kmem\_cache\_shrink( )** from shrinking a cache while another kernel control path attempts to allocate a new slab for it.

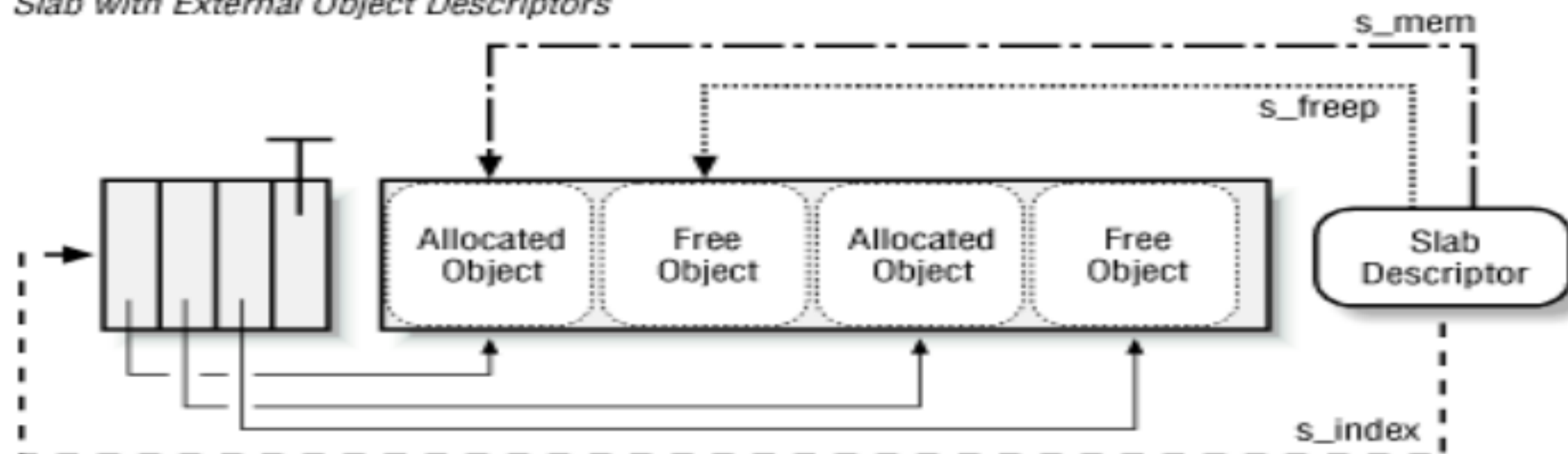
# Object descriptor

- Each object has a descriptor of type `struct kmem_bufctl_s`.
- The object descriptors of a slab can be stored in two possible ways,
- External Object descriptor - Stored outside the slab, in one of the general caches pointed to by `cache_sizes`. The first object descriptor in the memory area describes the first object in the slab and so on. The size of the memory area, and the general cache used to store object descriptors, depends on the number of objects stored in the slab (`c_num` field of the cache descriptor). The cache containing the objects themselves is tied to the cache containing their descriptors through two fields. First, the **`c_index_cache`** field of the cache containing the slab points to the cache descriptor of the cache containing the object descriptors. Second, the `s_index` field of the slab descriptor points to the memory area containing the object descriptors.
- Internal Object descriptor - Stored inside the slab, right after the objects they describe. In this case, the **`c_index_cache`** field of the cache descriptor and the `s_index` field of the slab descriptor are both NULL.

*Slab with Internal Object Descriptors*



*Slab with External Object Descriptors*



- The slab allocator chooses the first solution when the size of the objects is a multiple of 512, 1024, 2048, or 4096: in this case, storing control structures inside the slab would result in a high level of internal fragmentation.
- If the size of the objects is smaller than 512 bytes or not a multiple of 512, 1024, 2048, or 4096 the slab allocator stores the object descriptors inside the slab.
- Object descriptors are simple structures consisting of a single field:

```
typedef struct kmem_bufctl_s {  
    union {  
        struct kmem_bufctl_s * buf_nextp;  
        kmem_slab_t * buf_slabp;  
        void * buf_objp;  
    } u;  
} kmem_bufctl_t;  
#define buf_nextp u.buf_nextp  
#define buf_slabp u.buf_slabp  
#define buf_objp u.buf_objp
```

- This field has the following meaning, depending on the state of the object and the locations of the object descriptors:
- `buf_nextp` - If the object is free, it points to the next free object in the slab, thus implementing a simple list of free objects inside the slab.
- `buf_objp` - If the object is allocated and its object descriptor is stored outside of the slab, it points to the object.
- `buf_slabp` - If the object is allocated and its object descriptor is stored inside the slab, it points to the slab descriptor of the slab in which the object is stored. This holds whether the slab descriptor is stored inside or outside of the slab.

# Aligning Objects in Memory

- The objects managed by the slab allocator can be aligned in memory, that is, they can be stored in memory cells whose initial physical addresses are multiples of a given constant, usually a power of 2.
- This constant is called the alignment factor, and its value is stored in the **c\_align** field of the cache descriptor. The **c\_offset** field, which contains the object size, takes into account the number of padding bytes added to obtain the proper alignment. If the value of **c\_align** is 0, no alignment is required for the objects.
- The largest alignment factor allowed by the slab allocator is 4096, that is, the page frame size. This means that objects can be aligned by referring either to their physical addresses or to their linear addresses: in both cases, only the 12 least significant bits of the address may be altered by the alignment.



- When creating a new cache, it's possible to specify that the objects included in it be aligned in the first-level cache. To achieve this, set the **SLAB\_HWCACHE\_ALIGN** cache descriptor flag. The **kmem\_cache\_create()** function handles the request as follows:
- If the object's size is greater than half of a cache line, it is aligned in RAM to a multiple of **L1\_CACHE\_BYTES**, that is, at the beginning of the line.
- Otherwise, the object size is rounded up to a factor of **L1\_CACHE\_BYTES**; this ensures that an object will never span across two cache lines.
- Clearly, what the slab allocator is doing here is trading memory space for access time: it gets better cache performance by artificially increasing the object size, thus causing additional internal fragmentation.

# Slab Coloring

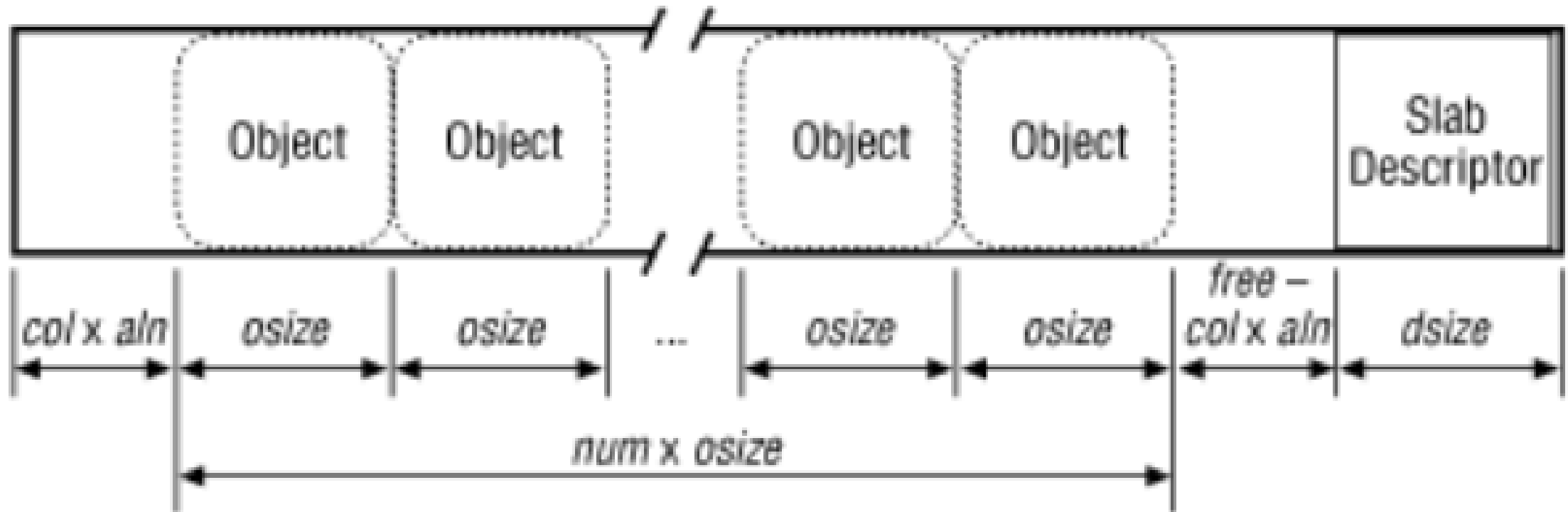
- The objects of the same size tend to be stored at the same offset within a cache.
- Objects that have the same offset within different slabs will, with a relatively high probability, end up mapped in the same cache line.
- The cache hardware might therefore waste memory cycles transferring two objects from the same cache line back and forth to different RAM locations, while other cache lines go underutilized.
- The slab allocator tries to reduce this unpleasant cache behavior by a policy called slab coloring: different arbitrary values called colors are assigned to the slabs.
- Before examining slab coloring, we have to look at the layout of objects in the cache.
- Let us consider a cache whose objects are aligned in RAM.
- Thus, the `c_align` field of the cache descriptor has a positive value, say `aln`. Even taking into account the alignment constraint, there are many possible ways to place objects inside the slab.
- The choices depend on decisions made for the following variables:
- `num` - Number of objects that can be stored in a slab. (value stored in the `c_num` field of the cache descriptor)

- `osize` - Object size including the alignment bytes (its value is in the `c_offset` field) plus object descriptor size (if the descriptor is contained inside the slab).
- `dsize` - Slab descriptor size; its value is equal to if the slab descriptor is stored outside of the slab.
- `free` - Number of unused bytes (bytes not assigned to any object) inside the slab.
- The total length in bytes of a slab can then be expressed as:

$$\text{slab length} = (\text{num} \times \text{osize}) + \text{dsize} + \text{free}$$

- `free` is always smaller than `osize`, since otherwise it would be possible to place additional objects inside the slab. However, `free` could be greater than `aln`.
- The slab allocator takes advantage of the free unused bytes to color the slab.
- The term "color" is used simply to subdivide the slabs and allow the memory allocator to spread objects out among different linear addresses.
- In this way, the kernel obtains the best possible performance from the microprocessor's hardware cache.

- Slabs having different colors store the first object of the slab in different memory locations, while satisfying the alignment constraint. The number of available colors is  $\text{free}/\text{aln}+1$ .
- The first color is denoted as  $\text{col}$  and the last one (whose value is in the  $\text{c\_colour}$  field of the cache descriptor) is denoted as  $\text{free}/\text{aln}$ .
- If a slab is colored with color  $\text{col}$ , the offset of the first object (with respect to the slab initial address) is equal to  $\text{col} \times \text{aln}$  bytes; this value is stored in the  $\text{s\_offset}$  field of the slab descriptor.



- Coloring works only when free is large enough. Clearly, if no alignment is required for the objects or if the number of unused bytes inside the slab is smaller than the required alignment ( $\text{free} < \text{aln}$ ), the only possible slab coloring is the one having the color 0, that is, the one that assigns a zero offset to the first object.
- The various colors are distributed equally among slabs of a given object type by storing the current color in a field of the cache descriptor called **c\_colour\_next**. The **kmem\_cache\_grow( )** function assigns the color specified by **c\_colour\_next** to a new slab and then decrements the value of this field. After reaching 0, it wraps around again to the maximum available value:

```
if (!(offset = cachep->c_colour_next--))
    cachep->c_colour_next = cachep->c_colour;
offset *= cachep->c_align;
slabp->s_offset = offset;
```

- In this way, each slab is created with a different color from the previous one, up to the maximum available colors.

# Allocating an Object into a Cache

- New objects may be obtained by invoking the **kmem\_cache\_alloc( )** function.
- The parameter **cachep** points to the cache descriptor from which the new free object must be obtained.
- **kmem\_cache\_alloc( )** first checks whether the cache descriptor exists; it then retrieves from the **c\_freep** field the address of the **s\_nextp** field of the first slab that includes at least one free object:

```
slabp = cachep->c_freep;
```

- If **slabp** does not point to a slab, it then jumps to **alloc\_new\_slab** and invokes **kmem\_cache\_grow( )** to add a new slab to the cache:

```
if (slabp->s_magic != SLAB_MAGIC_ALLOC)
```

```
goto alloc_new_slab;
```

- The value **SLAB\_MAGIC\_ALLOC** in the **s\_magic** field indicates that the slab contains at least one free object.
- If the slab is full, **slabp** points to the **cachep->c\_offset** field, and thus **slabp->s\_magic** coincides with **cachep->c\_magic**: in this case, however, this field contains a magic number for the cache different from **SLAB\_MAGIC\_ALLOC**.

- After obtaining a slab with a free object, the function increments the counter containing the number of objects currently allocated in the slab:

```
slabp->s_inuse++;
```

- It then loads **bufp** with the address of the first free object inside the slab and, correspondingly, updates the **slabp->s\_freep** field of the slab descriptor to point to the next free object:

```
bufp = slabp->s_freep;
```

```
slabp->s_freep = bufp->buf_nextp;
```

- If **slabp->s\_freep** becomes NULL, the slab no longer includes free objects, so the **c\_freepfield** of the cache descriptor must be updated:

```
if (!slabp->s_freep)
```

```
    cachep->c_freep = slabp->s_nextp;
```

- If the **slabp->s\_index** field is null, the object descriptors are stored right after the objects inside the slab.
- In this case, the address of the slab descriptor is first stored in the object descriptor's single field to denote the fact that the object is no longer free; then the object address is derived by subtracting from the address of the object descriptor the object size included in the **cachep->c\_offset** field:

```
if (!slabp->s_index) {  
    bufp->buf_slabp = slabp;  
    objp = ((void*)bufp) - cachep->c_offset;  
}
```

- If the **slabp->s\_index** field is not zero, it points to a memory area outside of the slab where the object descriptors are stored.
- In this case, the function first computes the relative position of the object descriptor in the outside memory area; it then multiplies this number by the object size; finally, it adds the result to the address of the first object in the slab, thus yielding the address of the object to be returned.
- As in the previous case, the object descriptor single field is updated and points now to the object:

```
if (slabp->s_index) {  
    objp = ((bufp-slabp->s_index)*cachep->c_offset) +  
    slabp->s_mem;  
    bufp->buf_objp = objp;  
}
```

- The function terminates by returning the address of the new object:  
 return objp;



# Releasing an Object from a Cache

- The `kmem_cache_free( )` function releases an object previously obtained by the slab allocator. Its parameters are **cachep**, the address of the cache descriptor, and **objp**, the address of the object to be released. The function starts by checking the parameters, after which it determines the address of the object descriptor and that of the slab containing the object.
- It uses the **cachep->c\_flags** flag, included in the cache descriptor, to determine whether the object descriptor is located inside or outside of the slab.
- In the former case, it determines the address of the object descriptor by adding the object's size to its initial address. The address of the slab descriptor is then extracted from the appropriate field in the object descriptor:

```
if (!SLAB_BUFCTL(cachep->c_flags)) {  
    bufp = (kmem_bufctl_t *)(objp+cachep->c_offset);  
    slabp = bufp->buf_slabp;  
}
```

- In the latter case, it determines the address of the slab descriptor from the prev field of the descriptor of the page frame containing the object.
- The address of the object descriptor is derived by first computing the sequence number of the object inside the slab (object address minus first object address divided by object length).
- This number is then used to determine the position of the object descriptor starting from the beginning of the outside area pointed to by the **slabp->s\_index** field of the slab descriptor.
- To be on the safe side, the function checks that the object's address passed as a parameter coincides with the address that its object descriptor says it should have:

```
if (SLAB_BUFCTL(cachep->c_flags)) {  
    slabp = (kmem_slab_t *)((&mem_map[MAP_NR(objp)])->prev);  
    bufp = &slabp->s_index[(objp - slabp->s_mem) /  
        cachep->c_offset];  
    if (objp != bufp->buf_objp)  
        goto bad_obj_addr;  
}
```

- Now the function checks whether the **slabp->s\_magic** field of the slab descriptor contains the correct magic number and whether the **slabp->s\_inuse** field is greater than 0.
- If everything is okay, it decrements the value of **slabp->s\_inuse** and inserts the object into the slab list of free objects:

```
slabp->s_inuse--;  
bufp->buf_nextp = slabp->s_freep;  
slabp->s_freep = bufp;
```

- If **bufp->buf\_nextp** is NULL, the list of free objects includes only one element: the object that is being released. In this case, the slab was previously filled to capacity and it might be necessary to reinsert its slab descriptor in a new position in the list of slab descriptors.
- (Remember that completely filled slabs appear before slabs with some free objects in the partially ordered list.)
- This is done by the `kmem_cache_one_free( )` function:

```
if (!bufp->buf_nextp)  
    kmem_cache_one_free(cachep, slabp);
```

- If the slab includes other free objects besides the one being released, it is necessary to check whether all objects are free. As in the previous case, this would make it necessary to reinsert the slab descriptor in a new position in the list of slab descriptors. The move is done by the **kmem\_cache\_full\_free()** function:

```
    if (bufp->buf_nextp)
```

```
    if (!slabp->s_inuse)
```

```
        kmem_cache_full_free(cachep, slabp);
```

- The kmem\_cache\_free( ) function terminates here.

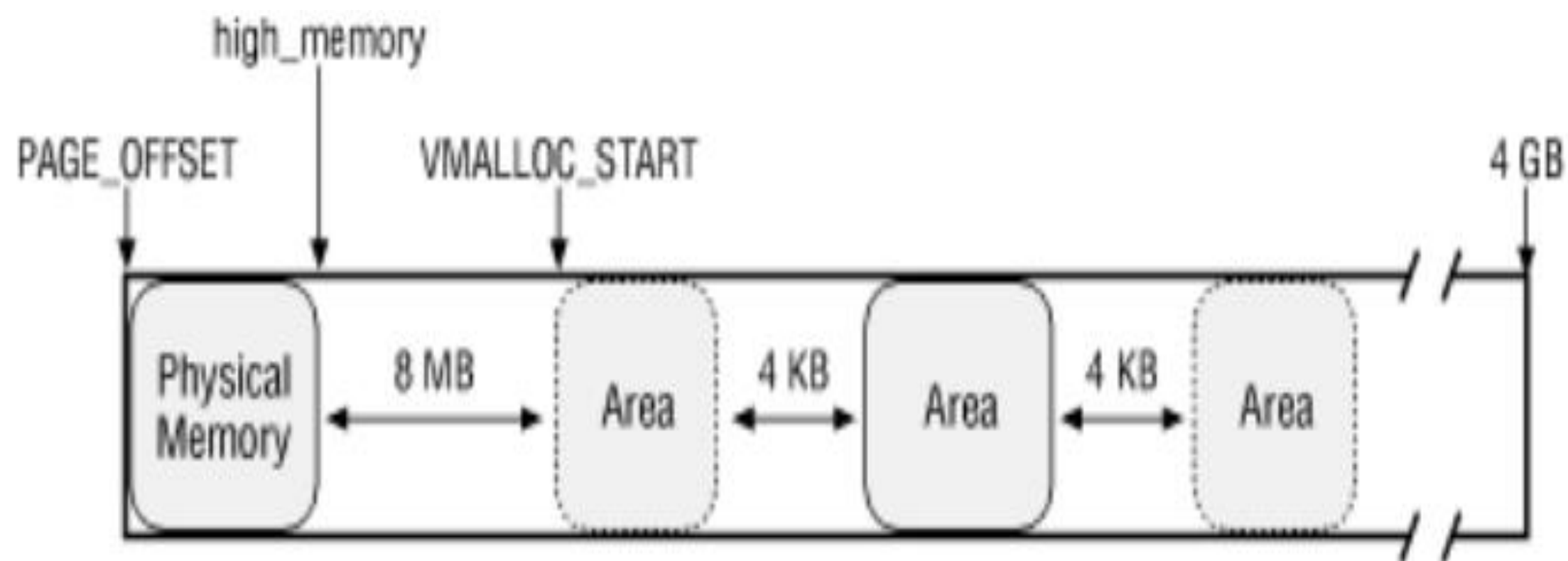
# Noncontiguous Memory Area Management

- It is preferable to map memory areas into sets of contiguous page frames, thus making better use of the cache and achieving lower average memory access times.
- Nevertheless, if the requests for memory areas are infrequent, it makes sense to consider an allocation schema based on noncontiguous page frames accessed through contiguous linear addresses.
- The main advantage of this schema is to avoid external fragmentation, while the disadvantage is that it is necessary to fiddle with the kernel page tables.
- Clearly, the size of a noncontiguous memory area must be a multiple of 4096.
- Linux uses noncontiguous memory areas sparingly, for instance, to allocate data structures for active swap areas, to allocate space for a module, or to allocate buffers to some I/O drivers.

# Linear Addresses of Noncontiguous Memory Areas

- To find a free range of linear addresses, we can look in the area starting from `PAGE_OFFSET`, i.e., the kernel reserved this whole upper area of memory to map available RAM for kernel use.
- The RAM occupies only a small fraction of the gigabyte, starting at the `PAGE_OFFSET` address.
- All the linear addresses above that reserved area are available for mapping noncontiguous memory areas.
- The linear address that corresponds to the end of physical memory is stored in the `high_memory` variable.
- A safety interval of size 8 MB (macro **`VMALLOC_OFFSET`**) is inserted between the end of the physical memory and the first memory area; its purpose is to "capture" out-of-bounds memory accesses.
- For the same reason, additional safety intervals of size 4 KB are inserted to separate noncontiguous memory areas.
- The **`VMALLOC_START`** macro defines the starting address of the linear space reserved for noncontiguous memory areas. It is defined as follows:

```
#define VMALLOC_START (((unsigned long) high_memory + \
VMALLOC_OFFSET) & ~(VMALLOC_OFFSET-1))
```



# Descriptors of Noncontiguous Memory Areas

- Each noncontiguous memory area is associated with a descriptor of type **struct vm\_struct**:

```
struct vm_struct {  
    unsigned long flags;  
    void * addr;  
    unsigned long size;  
    struct vm_struct * next;  
};
```

- These descriptors are inserted in a simple list by means of the next field; the address of the first element of the list is stored in the vmlist variable.
- The addr field contains the linear address of the first memory cell of the area.
- The size field contains the size of the area plus 4096 (the size of the previously mentioned interarea safety interval).



- The **get\_vm\_area( )** function creates new descriptors of type struct **vm\_struct**; its parameter size specifies the size of the new memory area.
- The function first calls **kmalloc( )** to obtain a memory area for the new descriptor.
- It then scans the list of descriptors of type struct vm\_struct looking for an available range of linear addresses that includes at least size+4096 addresses.
- If such an interval exists, the function initializes the fields of the descriptor and terminates by returning the initial address of the noncontiguous memory area.
- Otherwise, when addr + size exceeds the 4 GB limit, **get\_vm\_area( )** releases the descriptor and returns NULL.

# Allocating a Noncontiguous Memory Areas

- The **vmalloc( )** function allocates a noncontiguous memory area to the kernel.
- The parameter size denotes the size of the requested area.
- If the function is able to satisfy the request, then it returns the initial linear address of the new area; otherwise, it returns a NULL pointer.
- The function starts with size parameter, whose value has a multiple of 4096. It checks whether the size is greater than, less than or equal to existing number of page frames.
- If the size fits the available memory, the function invokes **vm\_area()**, which creates a new descriptor, and returns the linear addresses assigned to the memory area.
- **vmalloc( )** invokes **vmalloc\_area\_pages( )** to request noncontiguous page frames and terminates by returning the initial linear address of the noncontiguous memory area.
- The **vmalloc\_area\_pages( )** function makes use of two parameters: address, the initial linear address of the area, and size, its size. The linear address of the end of the area is assigned to the end local variable:

end = address + size;

- The cycle is repeated until all page table entries referring to the noncontiguous memory area have been set up.

# Releasing a Noncontiguous Memory Area

- The **vfree( )** function releases noncontiguous memory areas. Its parameter **addr** contains the initial linear address of the area to be released.
- **vfree( )** first scans the list pointed by **vmist** to find the address of the area descriptor associated with the area to be released.
- Each page frame assigned to the noncontiguous memory area is released by means of the buddy system **free\_page( )** function.
- The size field of the descriptor specifies the size of the area to be released. The area itself is released by invoking **vmfree\_area\_pages( )**, while the descriptor is released by invoking **kfree( )**.
- The **vmfree\_area\_pages( )** function takes two parameters: the initial linear address and the size of the area. It executes the following cycle to reverse the actions performed by **vmalloc\_area\_pages( )**.